

Reviewer Pack — Minimal Number of Algebraic Independence Relations and DPLL ...

Entry f0863ad599b6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 03:07:33 UTC

Minimal Number of Algebraic Independence Relations and DPLL Proof Tree Depth

Entry ID: f0863ad599b6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 03:07:33 UTC

1. Conjecture

Field A (mathematical branch): Model Theory (Algebraic Independence) **Field B** (complexity object): Complexity Theory (DPLL Search Trees)

Statement:

For every Boolean satisfiability problem, the minimal number of algebraic independence relations required to describe its DPLL proof tree depth is linearly correlated with its resolution width, i.e., the minimal number of independence relations, $m(n)$, satisfies $m(n) = O(w(n))$ where $w(n)$ is the resolution proof width.

Rationale (proposer's reasoning):

The study of algebraic independence in model theory can reveal hidden structures in Boolean satisfiability problems. If a direct relationship exists between the number of independence relations and DPLL proof tree depth, it may expose an underlying algebraic structure that could be exploited to design more efficient proof procedures or provide new insights into proof complexity.

Taxonomy category: algebraic_independence (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 272a24e1ac128b40

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the generated Boolean formulas have a linear relationship between the number of algebraic independence relations ($m(n)$) and resolution width ($w(n)$), measured by Pearson correlation coefficient ≥ 0.8 , with p-value ≤ 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - model theory algebraic independence AND DPLL proof tree depth - algebraic independence relations AND resolution proof width - DPLL search trees AND minimal number of independence relations

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_formula(n):
    clauses = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        if all(clause[i] != -clause[j] for j in range(i)):
            clauses.append(clause)
    return clauses

def dpll(clauses, assignment):
    unsatisfied_clauses = [c for c in clauses if not any(l in assignment and assignment[l] == v for l, v in c)]
    if not unsatisfied_clauses:
        return True
    literal = random.choice([l for c in unsatisfied_clauses for l in c if l not in assignment])
    if dpll(clauses, {**assignment, literal: 1}):
        return True
    if dpll(clauses, {**assignment, literal: -1}):
        return True
    return False

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    clauses = generate_formula(n)

    if not clauses:
        return {
```

```

        "metric_name": "resolution_width",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

resolution_width = len(clauses)
dpll_depth = 1

def backtrack(assignment, depth):
    nonlocal dpll_depth
    if not assignment:
        dpll_depth = max(dpll_depth, depth)
    for literal in range(1, n + 1):
        if literal not in assignment and -literal not in assignment:
            new_assignment = {**assignment, literal: 1}
            if dpll(clauses, new_assignment):
                backtrack(new_assignment, depth + 1)
            new_assignment = {**assignment, literal: -1}
            if dpll(clauses, new_assignment):
                backtrack(new_assignment, depth + 1)

backtrack({}, 0)

m_n = len([c for c in clauses if any(l in assignment and assignment[l] == v for l, v in zip(c, [-1, 1]))])

return {
    "metric_name": "resolution_width",
    "metric_value": resolution_width,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": m_n <= resolution_width,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        result = f"SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}"
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        result = f"FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}"

    print(result)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2cec965.py", line 87, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2cec965.py", line 40, in run_trial
    clauses = generate_formula(n)
              ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2cec965.py", line 22, in generate_form
    if all(clause[i] != -clause[j] for j in range(i)):
                                   ^
```

NameError: name 'i' is not defined. Did you mean: 'id'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient and p-value for at least 80% of the generated Boolean formulas as required by the support condition. | next: Review and fix the error in the test code to ensure it can complete without crashing and produce the necessary data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14921
2	preregistration	ollama_remote	glm4:latest	0	0	9826
3	novelty	ollama_remote	glm4:latest	0	0	8312
4	novelty	ollama_remote	glm4:latest	0	0	8914
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18787
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11518
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17382
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9300
9	judge	ollama_remote	glm4:latest	0	0	11797

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110757 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f0863ad599b6.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f0863ad599b6.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f0863ad599b6.tar.gz (if generated)